Scout

A company research and opportunity intelligence system for one serious job seeker.
Product requirements document, version 1.0, 13 November 2026.

> **What this is**
>
> You forward a recruiter email to an address. Four minutes later you have a sourced due-diligence report on that company, a fit score against your own profile, a list of red flags, and eight questions to ask in the interview. Every opportunity you ever log stays in one place, so after forty of them the system can also tell you what the market actually wants, what it keeps calling you, and which recruiters are worth answering.
>
> **The one-line scope test:** if a feature does not either (a) make a go / no-go decision faster, or (b) get better over time as more opportunities accumulate, it does not belong in v1.

Read this first: three honest points

Point	Detail
You already have a side project	Beacon, the accessibility agent, is your launchable product. Two unfinished side projects is the normal way both die. Scout should be scoped as a **personal tool that doubles as a portfolio artifact**, not as a second SaaS attempt. The roadmap below is built to be genuinely useful at the end of one weekend, not one quarter.
The tracker market is crowded, the research agent is not	Teal, Huntr, Simplify and Careerflow already do job tracking well, and that part is a commodity. The part nobody does well is **evidence-backed company due diligence on inbound roles**. If Scout ever becomes a product, that is the wedge. Build that first, and build the tracker around it.
Build the agent before the UI	The instinct for a designer is to build the dashboard first. Resist it. The agent is the only hard part and the only part that can fail. Phase 0 is a script with a JSON contract and no interface. The UI then sits on that contract, so none of the design work is wasted.

As a portfolio piece this is unusually well aimed at you. It is an agentic product with genuine uncertainty, provenance, cost and latency problems to solve in the interface, which is exactly the Google PAIR and Microsoft HAX territory you already work in. Section 10 lists the patterns worth building deliberately so the case study writes itself.

1. Problem

Each recruiter email is a decision under uncertainty with an asymmetric cost. Saying yes to a bad company costs weeks of interviews and possibly a wrong career move. Saying no to a good one costs an opportunity. Doing it properly takes two to four hours of research per company, which does not scale across a stream of inbound mail, so in practice the decision gets made on vibes, brand recognition, or whichever recruiter followed up hardest.

There is a second, slower problem. Every JD that arrives is a data point about what the market wants and how the market sees you. Today those data points arrive in an inbox and are lost. After forty of them there is a real signal in there: which skills keep appearing, what level you keep getting approached at, which channels produce good roles, where your funnel leaks.

Primary user

One person: you. Design for a single opinionated user with strong preferences, not for a generic audience. Multi-tenancy is a cheap structural decision to make on day one (section 11) but no product decision should be made for a hypothetical second user.

Success means

Outcome	Measure
Research effort collapses	Median time from email arriving to a reasoned go / no-go, under 10 minutes of your attention.
Nothing is lost	90%+ of inbound opportunities get logged, including the ones you reject immediately.
Decisions improve	You can name, for every company you interviewed with, the two risks you knew before the first call.
The data compounds	By opportunity 30, the trends view tells you something you did not already know.
It survives contact	You are still using it three months after you build it. This is the real test.

2. Product principles

These are the decisions that everything else follows from. Changing one of these changes the product; changing a feature does not.

#	Principle	Consequence
1	Intake must be nearly free	If logging an opportunity costs more than ten seconds you will only log the serious ones, and the trend analytics die from survivorship bias. Email forwarding is therefore not a nice-to-have, it is the feature that makes section 8 possible.
2	Company and Opportunity are different objects	Research the company once, version it, cache it, attach many roles to it. Saves cost, enables a watchlist, and means the second role from the same company is nearly free.
3	The agent returns structure, not prose	A typed object (ownership, layoff events, review scores, leadership changes, risk flags) with the narrative generated from it. Prose-only output gives you a pile of PDFs and no dashboard, no filtering, no trends, no comparison.
4	Evidence or it did not happen	Every material claim carries a source URL, a retrieved-at date and a quoted snippet. A claim with no evidence is dropped or explicitly labelled unverified. This is the only real defence against confident wrong facts about real companies, and it is what made the Sonatype report usable.
5	Fit is relative to you, and your weights move	Your profile and your scoring weights are first-class editable objects. Job security matters more on a contract than it will in two years. Scores recompute on demand.
6	Freshness is part of the answer	Every report shows its age. Companies change. Re-run before each interview round, and diff against the previous run so you can see what moved.
7	Confidence is displayed, never hidden	Three levels: confirmed (primary source), reported (credible secondary), unverified (single weak source). The interface shows which is which. Never present an inference as a fact.

3. Scope

In scope for v1

- Email, paste, file and URL intake of JDs
- Deep company research agent with evidence
- Company profiles, versioned and re-runnable
- Opportunity records with parsed JD fields
- Fit and risk scoring against your profile
- Dashboard, filters, side-by-side comparison
- Pipeline stages and interview scheduling
- Reminders and a weekly digest
- Trend analytics across all logged JDs
- Interview prep pack generation
- PDF export of any report

Explicitly out

- Scraping Glassdoor, LinkedIn, AmbitionBox or Blind (section 13)
- Applying to jobs on your behalf
- Resume tailoring and cover letter generation; that is a different product
- Job board aggregation or search; this is inbound-only by design
- Salary data purchase or paid data vendors in v1
- Mobile apps; responsive web is enough
- Any feature justified by a second user who does not exist

4. Core concepts and data model

Object	What it is	Lifecycle
CandidateProfile	You: experience, skills, projects, location, constraints, scoring weights.	Edited rarely, versioned.
Company	A canonical company, keyed on domain. One row per real company.	Permanent, deduped.
ResearchRun	One execution of the agent against a Company. Holds the structured payload, cost, model, duration, status.	Append-only; newest is current; diffable.
Evidence	A source: URL, publisher, title, retrieved_at, snippet. Claims reference it.	Append-only.
Opportunity	A specific role at a Company, from a specific source, at a point in time.	Moves through stages.
Requirement	One extracted requirement from a JD, normalised to the skill taxonomy.	Derived, recomputable.
Score	One dimension of one Opportunity, with rationale and the evidence behind it.	Recomputed on weight change.
Contact	A recruiter or interviewer, with agency and history.	Permanent.
InterviewRound	A scheduled round: time, format, interviewers, prep state, outcome, notes.	Scheduled, then closed.
Activity	Any timeline event: email received, research done, stage change, round held.	Append-only.

Schema sketch

```
companies          id, domain UNIQUE, legal_name, display_name, hq_country, hq_city,
                   founded_year, ownership_type, website, linkedin_url, created_at

research_runs      id, company_id FK, status, payload JSONB, schema_version, model,
                   input_tokens, output_tokens, searches_used, cost_usd, duration_ms,
                   started_at, completed_at, error, superseded_by FK
```

```
evidence           id, research_run_id FK, url, domain, title, publisher,
                   published_at, retrieved_at, snippet, trust_tier

opportunities      id, user_id FK, company_id FK, title, level_claimed, level_inferred,
                   location, work_policy, employment_type, comp_min, comp_max, currency,
                   source_channel, contact_id FK, raw_text, parsed JSONB, stage,
                   received_at, decided_at, decision, decision_note

requirements       id, opportunity_id FK, raw_phrase, skill_id FK, category, is_required
skills             id, canonical_name, aliases TEXT[], category
scores             id, opportunity_id FK, dimension, value NUMERIC, rationale,
                   evidence_ids UUID[], computed_at, weights_version

contacts           id, user_id FK, name, email, agency, company_id FK, notes
interview_rounds   id, opportunity_id FK, seq, kind, scheduled_at, duration_min,
                   interviewers JSONB, prep_pack_id FK, outcome, notes
activities         id, user_id FK, entity_type, entity_id, kind, payload JSONB, at
```

Put the research payload in JSONB behind a `schema_version` and validate it with Zod on read. You will change that shape five times in the first month, and a normalised table per fact type would make every change a migration. Normalise later, once the shape stops moving, starting with the fields the trends view needs to query.

5. The research agent

This is the product. Everything else is furniture around it.

5.1 Pipeline

Eight stages. Stage 0 and 1 run first and in sequence. Stages 2 to 7 run in parallel. Stage 8 depends on all of them. Each stage has its own prompt, its own output schema, and its own search budget, so a failure in one stage degrades the report instead of killing it.

#	Stage	Produces	Budget
0	Intake parse	company_name, domain guess, role title, level, location, recruiter, comp hint, raw text	0 searches
1	Entity resolution	The actual company: legal name, domain, HQ, founded, LinkedIn, public or private	2 to 4
2	Business	What it sells, products, customers, segments, scale claims	4 to 6
3	Money	Funding rounds, ownership, PE or VC, M&A;, revenue and profitability signals, sale or IPO chatter	5 to 8
4	People	Headcount trend, layoff events with dates and sizes, leadership changes, review scores by source and their direction	6 to 10
5	Market	Competitors, analyst placements, moat, structural threats	4 to 6
6	Local entity	India or regional site: age, headcount, charter (product vs support vs cost centre), office, work policy	3 to 5
7	Role context	Is the role posted publicly, at what level, how long open, how many similar roles, comp benchmarks	3 to 5
8	Synthesis	Risk flags, green flags, scores, questions to ask, verdict, narrative	0 searches

Stage 1 is the one everyone skips and the one that bites

While researching Sonatype, searches returned Sonata Software, Sonatus and Sonix as well. Three different companies, all plausible, all wrong. If the agent silently picks the wrong one the entire report is confidently about somebody else. Stage 1 must therefore return a confidence score, and below a threshold it must stop and ask you to pick from candidates rather than guess. That hand-back is a product requirement, not an edge case.

5.2 Output contract

Every fact is an object, never a bare string. This single decision is what makes the dashboard, the filters, the trends and the comparison view possible later.

```
type Claim<T> = {
  value: T
  confidence: 'confirmed' | 'reported' | 'unverified'
  evidence: EvidenceRef[]        // >= 1 required, or the claim is dropped
  as_of: string | null           // what date this value describes
  note?: string
}

type CompanyResearch = {
  schema_version: 2
  identity:  { legal_name: Claim<string>; domain: string; hq: Claim<string>; ... }
  business:  { what_they_sell: Claim<string>; products: Claim<string[]>; ... }
  money:     { ownership_type: Claim<'public'|'pe_backed'|'vc_backed'|'bootstrapped'|'unknown'>
               funding_events: Claim<FundingEvent>[]; revenue_signal: Claim<string> | null
               profitability: Claim<string> | null; exit_activity: Claim<string>[] }
  people:    { headcount: Claim<number> | null; headcount_trend: Claim<TrendPoint[]>
               layoff_events: Claim<LayoffEvent>[]      // date, size, pct, handling notes
               leadership_changes: Claim<LeadershipChange>[]
```

```
                review_scores: Claim<ReviewScore>[] }    // source, score, n, trend
  market:     { competitors: Claim<string[]>; analyst_position: Claim<string>[]
                structural_risks: Claim<string>[] }
  local:      { has_site: boolean; site_age_months: Claim<number> | null
                charter: Claim<'product'|'support'|'mixed'|'unknown'>; ... }
  role:       { posted_publicly: Claim<boolean>; public_level: Claim<string> | null
                days_open: Claim<number> | null; similar_openings: Claim<number> | null }
  flags:      Flag[]              // {kind, severity 1-5, title, detail, evidence}
  gaps:       string[]            // what could NOT be established, stated plainly
}
```

The `gaps` array matters more than it looks. A report that silently omits what it could not find reads as more certain than it is. Listing the gaps makes the report honest and tells you what to ask in the call.

5.3 Flag taxonomy

A fixed vocabulary, not free text. Fixed means you can filter, count, chart and compare it.

Risk flags

- **layoff_history** rounds, dates, sizes, how they were handled
- **pe_ownership** plus sale or recap process in motion
- **leadership_churn** C-suite changes inside 18 months
- **sentiment_decline** review scores trending down
- **cost_centre_risk** local site is support, not product
- **site_immaturity** local entity under 18 months old
- **role_ambiguity** rotation, contract, or two JDs merged
- **level_mismatch** JD asks well above or below you
- **funding_risk** down round, long gap, visible burn
- **market_commoditisation** core product being bundled away
- **attrition_signal** same role reposted repeatedly
- **rto_change** work policy recently tightened

Green flags

- **site_growth** local office actively expanding
- **profitable** credible profitability signal
- **category_leader** analyst-recognised position
- **durable_moat** asset competitors cannot copy
- **rare_role** few companies hire for this shape
- **product_charter** real product ownership locally
- **domain_fit** matches your stated interests
- **design_maturity** design has a seat and a ladder
- **public_comp_band** pay transparency in postings
- **tenure_signal** people stay, per public profiles

5.4 Scoring

Six dimensions, 0 to 100 each, each one traceable to the evidence that produced it. Weights are yours to set and every score recomputes when you change them. Never show a single blended number without its parts; a 72 that hides a 20 on job security is worse than no number.

Dimension	Driven by	Default weight
Role fit	JD requirements matched against your profile and projects	30%
Company health	Money stage: funding, revenue, profitability, market position	15%
Job security	Layoff history, ownership, headcount trend, local site charter and age	20%
Career signal	Title, level, scope, whether the role is rare or commodity	15%
Culture and sentiment	Review scores and their direction, leadership stability, tenure	10%
Comp confidence	Disclosure, benchmarks, local site pay reputation	10%

5.5 Guardrails

Control	Rule
Search budget	Hard cap per run via max_uses on the web search tool, per stage and in total. Default 40 searches.

Control	Rule
Token budget	Hard cap per run. Abort and return a partial report rather than running away.
Model tiering	Cheap fast model for parsing and extraction, mid tier for research stages, top tier for synthesis only. Read current model IDs from the Models API rather than hardcoding them.
Evidence enforcement	Schema validation rejects any claim with an empty evidence array. Validate before persisting, not after.
Domain trust tiers	Primary (company filings, press releases, government registries), secondary (established press, analyst firms), tertiary (aggregators, blogs). Tier is stored on the evidence and shown in the interface.
No fabricated numbers	Compensation, revenue and headcount are either sourced or absent. No estimates presented as data, ever.
Cost recorded per run	Displayed in the interface. You should always know what a report cost.
Eval harness	Five companies with hand-checked ground truth, run on every prompt change. You already work TDD on Beacon; apply the same discipline here, because prompt regressions are silent otherwise.

6. Feature set

6.1 Intake

Feature	Notes	Pri
Paste JD text	The fastest path to a working system. Build this first.	MUST
Email forwarding	A dedicated ingest address via an inbound email provider webhook. Parse sender, subject, body, attachments. This is the feature that makes the whole thing stick.	MUST
File upload	PDF and DOCX JDs. You already have the parsing toolchain.	SHOULD
URL intake	Paste a posting link, fetch and parse it.	SHOULD
Auto-extraction	Company, role, level, location, work policy, comp if stated, recruiter, agency.	MUST
Duplicate detection	Same company and similar title inside 90 days gets linked, not duplicated.	SHOULD
Browser extension	Clip from LinkedIn or Naukri. Nice, not necessary.	LATER

6.2 Research and profiles

Feature	Notes	Pri
Run deep research	The section 5 pipeline, triggered automatically on intake.	MUST
Live progress	Stage-by-stage status while it runs. A four-minute silent spinner is unacceptable.	MUST
Company profile page	Structured report with evidence inline and gaps stated.	MUST
Disambiguation prompt	When stage 1 is not confident, ask instead of guessing.	MUST
Re-run and diff	Re-run on demand; show what changed against the previous run.	SHOULD
Staleness indicator	Age shown everywhere a report appears. Prompt to re-run past 60 days.	SHOULD
PDF export	You think in PDFs. One click, structured, sourced.	SHOULD
Watchlist	Companies with no live role. Periodic re-check, alert on material news.	COULD

6.3 Pipeline and interviews

Feature	Notes	Pri
Stages	Inbox, Researching, Reviewed, Screening, Interviewing, Offer, Closed. Kanban and table views.	MUST
Decision capture	When you reject something, record why in one click from a fixed list. This is what feeds the trends view.	MUST
Interview rounds	Date, time, format, interviewers, notes, outcome.	MUST
Calendar	ICS export first, Google Calendar sync later.	SHOULD
Reminders	Prep 24h before, follow-up 2 days after, nudge after 7 days of silence.	SHOULD
Prep pack	Per round: likely questions, your questions, JD requirements mapped to your project evidence, gaps to prepare, news from the last 30 days.	SHOULD
Post-round notes	Structured capture while it is fresh, including your own read on them.	SHOULD
Offer comparison	CTC against in-hand, equity realism, notice, relocation, side by side.	COULD

6.4 Decision support

Feature	Notes	Pri
Scorecard	Six dimensions with rationale, on every opportunity.	MUST
Compare view	Two to four companies side by side on the same rows.	SHOULD
Weight editor	Change your priorities, everything recomputes.	SHOULD
Questions to ask	Generated per opportunity from the gaps and flags found.	MUST
Weekly digest	Email on Monday: new opportunities, decisions waiting, interviews this week, one trend observation.	COULD

7. Trend analytics

This only works if you log rejections too. Ten charts, specified concretely so they do not turn into a generic dashboard of vanity metrics.

#	Chart	Question it answers	Needs
1	Skill demand over time	Which skills keep appearing in JDs sent to me, and which are rising?	20 JDs
2	Skill gap ranking	Which in-demand skills are missing from my profile, weighted by how good those roles were?	20 JDs
3	Level distribution	What title and level is the market actually approaching me at? Senior, Lead, or Staff?	15 JDs
4	Comp band distribution	Where disclosed, what range am I being approached with, and is it moving?	10 with comp
5	Source quality	Which recruiters, agencies and channels bring roles that score well and progress far?	25 JDs
6	Funnel conversion	Inbound to screen to interview to offer, with conversion rates and median days per stage.	10 in pipeline
7	Response latency and ghosting	Who replies, how fast, and who disappears.	15 contacted
8	Company archetype mix	Product company, GCC, services, startup. What is actually reaching me?	20 companies
9	Red flag prevalence	What share of inbound companies carry layoff history or PE ownership? Is my inbound quality improving?	20 companies
10	Market temperature	Inbound volume per week over time, by archetype.	8 weeks

> **Design note.** Hide every chart until it has enough data to be honest. A trend line drawn through four points is a lie with a nice gradient. Show the locked state with its threshold instead: "needs 20 opportunities, you have 7." It sets expectations, it gives you a reason to keep logging, and it is a better interface than an empty chart.

8. Screens

Screen	Purpose	Key elements
Inbox	The daily surface. Everything that needs a decision.	Cards with company, role, score, top two flags, age. Quick actions: research, reject with reason, move to pipeline.
Opportunity detail	Where the go / no-go decision is made.	Scorecard with expandable rationale, flags, the JD with requirements highlighted and matched against you, the company report, questions to ask, action bar.

Screen	Purpose	Key elements
Company profile	Durable company knowledge.	Structured report by section, evidence inline and expandable, gaps panel, run history and diff, all roles ever seen from them.
Pipeline	Where things stand.	Kanban by stage, interview dates, next actions, idle-too-long warnings.
Compare	Choosing between live options.	Two to four columns, aligned rows, differences emphasised rather than repeated.
Trends	The slow payoff.	The ten charts, each with its locked state and the threshold to unlock.
Profile and weights	Tuning the system.	Your experience, skills, projects, constraints, scoring weights with live preview of what re-ranks.
Run inspector	Trust and debugging.	Stage timeline, searches run, sources used, tokens, cost, raw payload. Also your proof of craft when you write this up.

9. AI-UX patterns worth building deliberately

You sell yourself as an AI-UX specialist. This project gives you a genuine reason to solve these problems rather than describe them. Build them on purpose and the case study writes itself.

Pattern	The problem	What to build
Progress for slow agents	A four-minute wait with no signal feels broken.	Stage-level streaming: name the stage, show searches as they happen, show partial results filling in. Let the user read stage 2 while stage 5 runs.
Provenance at a glance	Users cannot trust claims they cannot check, but footnotes are ignored.	Inline confidence marks, hover for source and date, one click to the original. Make checking cheaper than doubting.
Uncertainty that is legible	Hedged prose reads as weak; false confidence is dangerous.	Three discrete confidence levels with distinct visual treatment, plus an explicit gaps panel. Discrete beats a percentage nobody can calibrate.
Graceful partial failure	One stage failing should not kill the report.	Per-stage status. Show what succeeded, mark what failed, offer a retry for that stage alone.
Disagreeing with the agent	The score will sometimes be wrong.	Let the user override any score with a reason, and keep both. Overrides are training data for your own prompt tuning.
Cost and effort made visible	Hidden cost erodes trust in agentic tools.	Show searches run, time taken and money spent per report. Unusual in consumer products, and memorable in a portfolio.
Change over time	A stale report is worse than none.	Diff view between runs, with material changes called out in words rather than a raw JSON delta.
Human in the loop at the right moment	Full autonomy fails at ambiguity.	The stage 1 disambiguation prompt. Interrupt once, at the only point where a wrong guess poisons everything downstream.

10. Technical architecture

Layer	Choice	Why
Frontend	Next.js App Router, TypeScript, Tailwind, shadcn/ui	Your existing stack. No learning tax.
Database and auth	Supabase: Postgres, Auth, Storage, row level security	RLS gives multi-tenancy almost free on day one, which keeps the product path open without any extra work now.
Background jobs	Inngest	The agent is a multi-step workflow with retries, parallel fan-out and a four-minute runtime. Step functions are the right shape and serverless timeouts are not.
LLM	Claude API with the web search server tool	Search runs server side with citations returned, so you do not build or pay for a separate search pipeline. Set max_uses per call to cap spend. Add the web fetch tool for full-page reads.
Validation	Zod schemas shared between agent output, database reads and UI props	One source of truth for the contract in section 5.2.
Inbound email	Postmark inbound, or Cloudflare Email Workers, to a webhook	Either works. Postmark parses attachments for you; Cloudflare is cheaper and more fiddly.
Charts	Recharts	Adequate, and you can style it to match.
PDF	Server-rendered HTML to PDF via Playwright, or React PDF	HTML to PDF keeps one set of styles for screen and print.

Layer	Choice	Why
Hosting	Vercel plus Supabase plus Inngest free tiers	All three have usable free tiers at single-user volume.

Flow

```
email -> webhook -> parse -> opportunity.created
                                    |
                          Inngest: research.requested
                                    |
                  stage 0 parse -> stage 1 resolve entity
                                    |
                       [confidence < threshold] -> pause, ask user
                                    |
          parallel: stage 2 | 3 | 4 | 5 | 6 | 7   (each cached, each budgeted)
                                    |
                        stage 8 synthesis + scoring
                                    |
                persist run + evidence -> revalidate -> notify
```

Cost control

Check current model pricing at platform.claude.com before you tune anything, and measure your own first ten runs rather than trusting an estimate. The levers that matter: cap searches with max_uses, tier the models so only synthesis uses the expensive one, cache the company research so a second role from the same company costs nothing, use prompt caching for the parts of the prompt that repeat across stages, and record cost per run so a regression is visible immediately. Build a monthly ceiling in with a hard stop, not a warning.

11. Data source constraints, read before building

> **Do not build a scraper.** Glassdoor, LinkedIn, AmbitionBox and Blind all prohibit automated collection in their terms, and all of them block it aggressively in practice. An account ban on LinkedIn would cost you considerably more than this tool is worth, given how you find work.
>
> The Sonatype research worked without touching any of them directly. It worked through search engine result snippets, which surface review aggregate scores and individual review text, plus fetches of pages that permit it: press releases, news articles, company sites, analyst summaries, job boards. That is the pattern to build on.

Source type	Approach	Reliability
Company press releases and newsroom	Fetch directly. Primary source.	High, but promotional
News and trade press	Search, then fetch the article.	High
Analyst placements (Gartner, Forrester)	Usually only visible through vendor press releases. Treat as vendor-reported.	Medium
Funding and ownership	Press releases, news, PE firm sites. Crunchbase and Tracxn via search snippets only.	Medium to high
Layoffs	News, layoffs.fyi, employee reviews surfaced in search results.	Medium
Review scores	Search result snippets, which generally expose aggregate scores and review text. Never scrape the sites.	Medium, and self-selected
Headcount	Workforce intelligence pages via search; company claims.	Low to medium, often conflicting
Compensation	Levels.fyi and similar via search. India data is usually thin to absent.	Low
Local entity details	Local press, government and trade announcements, careers pages.	Medium

Two things follow from this table. First, confidence levels are not decoration; most of what you want sits at medium reliability and the interface has to say so. Second, compensation data for India will mostly not exist, so the comp confidence score should usually read as unknown. Do not let the agent fill that silence with an estimate; job aggregators publish AI-generated salary guesses that look authoritative and are not evidence.

12. Risks

Risk	Likelihood	Mitigation
Confident wrong facts about real companies	High	Evidence enforced in the schema. Confidence shown. Gaps listed. Eval harness on five known companies run on every prompt change.
Wrong company researched entirely	Medium	Stage 1 disambiguation with a hand-back to you below a confidence threshold.
Cost runs away	Medium	Per-run search and token caps, model tiering, company-level caching, monthly ceiling with a hard stop.
Scope creep kills it	High	Phase 0 is useful on its own. Ship it before building any screen.
Abandoned after three weeks	High	This is the real risk. It competes with Beacon for the same evenings. Decide now which one gets the time.
Reports go stale and mislead	Medium	Age shown everywhere, re-run prompt past 60 days, diff view.

Risk	Likelihood	Mitigation
Review data is self-selected and negative-skewed	Certain	Always show sample size and direction of travel, never a bare score. Say so in the interface.

13. Roadmap

Each phase ends with something you would actually use. If you stop after any phase, you still have a working tool.

Phase	Scope	Done when	Effort
0. The agent	No UI. A TypeScript script: paste JD text in, get the section 5.2 JSON out, plus a markdown report. Stages 0, 1, 3, 4, 8 only. Cost logging. Eval set of five companies.	You run it on a real recruiter email and the output is as good as the Sonatype report.	1 weekend
1. Thin shell	Next.js plus Supabase. Paste intake, opportunity list, company profile page, scorecard, PDF export. Inngest running the agent with stage progress.	You use it instead of the script, on a live opportunity.	1 week
2. Zero-effort intake	Email forwarding, auto-extraction, duplicate detection, remaining research stages, disambiguation flow, re-run and diff.	You forward an email and do nothing else.	1 week
3. Pipeline	Stages, decision capture with reasons, interview rounds, ICS export, reminders.	You stop keeping interview dates anywhere else.	1 week
4. Compounding value	Trend analytics, compare view, weight editor, prep packs, weekly digest.	A chart tells you something you did not know.	1 to 2 weeks
5. Optional product path	Only if you still want it after using Scout for two months. Onboarding, billing, rate limits, shared company cache across users, which is the real economic advantage of a multi-user version.	Someone who is not you pays for it.	Open

Phase 0 is the whole bet. If the agent produces something materially worse than what careful manual research gives you, no amount of interface saves the product and you should stop. If it produces something close, everything after that is ordinary web development that you can already do. Spend the first weekend finding out which it is.

14. Decisions for you to make

#	Decision	My recommendation
1	Scout or Beacon for the next six weeks?	Beacon if you still want a launchable product, since Scout has a crowded market and a thin moat. Scout if you want a tool you will use weekly and a portfolio case study that proves your AI-UX positioning. Doing both halves both.
2	Single-user or product from day one?	Build single-user, but put Supabase auth and RLS in from the start. It costs perhaps two hours now and keeps the door open.
3	Which email provider for intake?	Postmark inbound. Attachment parsing is handled for you and the free tier covers your volume.
4	How deep should research go by default?	Start at roughly 40 searches and four minutes. Measure the cost on your first ten runs, then decide whether a cheaper quick mode is worth adding.
5	Public repo?	Yes, if you are using it as a portfolio piece. Keep the eval set and your candidate profile out of it.

15. What to name it

Scout is the working title in this document. It is short, it suggests reconnaissance ahead of a decision, and the domain space is crowded but the name reads well in a case study. Alternatives worth a look: Vetted, Recce, Duediligence.dev, Pre-Read, Backchannel. Backchannel is the most interesting of those, since it is exactly what this tool substitutes for: the friend inside the company who tells you what it is really like.

Companion file: **scout-claude-code-prompt.md** contains the kickoff prompt, a starter CLAUDE.md, the shared type contract and the Phase 0 task list, ready to paste into Claude Code.